

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №6
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Унарные и бинарные операции над графами»

Выполнили:
студенты группы 24ВВВЗ
Задоркин Тимофей
Комиссаров Андрей

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2025

Цель работы – Изучение и практическое освоение унарных и бинарных операций над графами, включая операции, уменьшающие или увеличивающие число элементов графа, такие как удаление и добавление вершин и рёбер, отождествление и расщепление вершин, стягивание рёбер.

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1, M_2 смежности неориентированных помеченных графов G_1, G_2 . Выведите сгенерированные матрицы на экран.

2. * Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2. * Для представления графов в виде списков смежности выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$

в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Задание 4 *

1. Для матричной формы представления графов выполните операцию декартова произведения графов $G = G_1 \times G_2$.

Результат выполнения операции выведите на экран.

Задание 1.

Код программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct Node {
    int vertex;
    struct Node* next;
};

struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

void generateAdjacencyMatrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else {
                int value = rand() % 2;
                matrix[i][j] = value;
                matrix[j][i] = value;
            }
        }
    }
}

void printMatrix(int** matrix, int n) {
    printf("    ");
    for (int i = 0; i < n; i++) {
        printf("%2d ", i + 1);
    }
    printf("\n");

    for (int i = 0; i < n; i++) {
```

```

        printf("%2d: ", i + 1);
        for (int j = 0; j < n; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

struct Node** matrixToList(int** matrix, int n) {
    struct Node** adjList = (struct Node**)malloc(n * sizeof(struct
Node*));

    for (int i = 0; i < n; i++) {
        adjList[i] = NULL;
        for (int j = 0; j < n; j++) {
            if (matrix[i][j] == 1 && i != j) {
                struct Node* newNode = createNode(j + 1);
                newNode->next = adjList[i];
                adjList[i] = newNode;
            }
        }
    }

    return adjList;
}

void printAdjacencyList(struct Node** adjList, int n) {
    for (int i = 0; i < n; i++) {
        printf(" %d: ", i + 1);
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            printf("%d ", temp->vertex);
            temp = temp->next;
        }
        printf("\n");
    }
}

void freeMatrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void freeAdjacencyList(struct Node** adjList, int n) {
    for (int i = 0; i < n; i++) {
        struct Node* temp = adjList[i];
        while (temp != NULL) {
            struct Node* toDelete = temp;
            temp = temp->next;
            free(toDelete);
        }
    }
    free(adjList);
}

```

```

int main() {
    system("chcp 1251");
    srand(time(NULL));

    int n;
    printf("Введите количество вершин в графах: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Количество вершин должно быть положительным числом.\n");
        return 1;
    }

    int** M1 = (int**)malloc(n * sizeof(int*));
    int** M2 = (int**)malloc(n * sizeof(int*));

    for (int i = 0; i < n; i++) {
        M1[i] = (int*)malloc(n * sizeof(int));
        M2[i] = (int*)malloc(n * sizeof(int));
    }

    printf("\n=== ГЕНЕРАЦИЯ МАТРИЦ СМЕЖНОСТИ ===\n");

    printf("\nМатрица смежности M1 (граф G1):\n");
    generateAdjacencyMatrix(M1, n);
    printMatrix(M1, n);

    printf("\nМатрица смежности M2 (граф G2):\n");
    generateAdjacencyMatrix(M2, n);
    printMatrix(M2, n);

    printf("\n=== ПРЕОБРАЗОВАНИЕ В СПИСКИ СМЕЖНОСТИ ===\n");

    struct Node** adjList1 = matrixToList(M1, n);
    struct Node** adjList2 = matrixToList(M2, n);

    printf("\nСписок смежности для графа G1:\n");
    printAdjacencyList(adjList1, n);

    printf("\nСписок смежности для графа G2:\n");
    printAdjacencyList(adjList2, n);

    freeMatrix(M1, n);
    freeMatrix(M2, n);
    freeAdjacencyList(adjList1, n);
    freeAdjacencyList(adjList2, n);

    return 0;
}

```

Задание 2.

Код программы

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

```

```

#include <stdlib.h>
#include <stdbool.h>

typedef struct {
    int** matrix;
    int vertices;
} GraphMatrix;

GraphMatrix* createGraphMatrix(int vertices) {
    GraphMatrix* graph = (GraphMatrix*)malloc(sizeof(GraphMatrix));
    graph->vertices = vertices;
    graph->matrix = (int**)malloc(vertices * sizeof(int*));

    for (int i = 0; i < vertices; i++) {
        graph->matrix[i] = (int*)calloc(vertices, sizeof(int));
    }
    return graph;
}

void printGraphMatrix(GraphMatrix* graph) {
    printf("Матрица смежности:\n");
    for (int i = 0; i < graph->vertices; i++) {
        for (int j = 0; j < graph->vertices; j++) {
            printf("%d ", graph->matrix[i][j]);
        }
        printf("\n");
    }
}

void addEdgeMatrix(GraphMatrix* graph, int src, int dest) {
    graph->matrix[src - 1][dest - 1] = 1;
    graph->matrix[dest - 1][src - 1] = 1;
}

void identifyVerticesMatrix(GraphMatrix* graph, int v1, int v2) {
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;

    if (v1 == v2 || idx1 >= graph->vertices || idx2 >= graph->vertices
    || idx1 < 0 || idx2 < 0) {
        printf("Неверные вершины!\n");
        return;
    }

    int newVertices = graph->vertices - 1;
    int** newMatrix = (int**)malloc(newVertices * sizeof(int*));
    for (int i = 0; i < newVertices; i++) {
        newMatrix[i] = (int*)calloc(newVertices, sizeof(int));
    }

    for (int i = 0; i < graph->vertices; i++) {
        for (int j = 0; j < graph->vertices; j++) {
            if (i == idx2 || j == idx2) continue;

            int newI = i;
            int newJ = j;

            if (i > idx2) newI = i - 1;

```

```

        if (j > idx2) newJ = j - 1;

        newMatrix[newI][newJ] = graph->matrix[i][j];
    }
}

for (int i = 0; i < graph->vertices; i++) {
    if (i != idx1 && i != idx2) {
        int newI = i;
        if (i > idx2) newI = i - 1;

        if (graph->matrix[idx2][i]) {
            newMatrix[idx1][newI] = 1;
            newMatrix[newI][idx1] = 1;
        }
    }
}

for (int i = 0; i < graph->vertices; i++) {
    free(graph->matrix[i]);
}
free(graph->matrix);

graph->matrix = newMatrix;
graph->vertices = newVertices;
}

void contractEdgeMatrix(GraphMatrix* graph, int v1, int v2) {
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;

    if (v1 == v2 || idx1 >= graph->vertices || idx2 >= graph->vertices
|| idx1 < 0 || idx2 < 0) {
        printf("Неверные вершины!\n");
        return;
    }

    if (graph->matrix[idx1][idx2] == 0) {
        printf("Ребро между вершинами %d и %d не существует!\n", v1,
v2);
        return;
    }

    identifyVerticesMatrix(graph, v1, v2);
}

void splitVertexMatrix(GraphMatrix* graph, int v) {
    int idx = v - 1;

    if (idx >= graph->vertices || idx < 0) {
        printf("Неверная вершина!\n");
        return;
    }

    int newVertices = graph->vertices + 1;
    int** newMatrix = (int**)malloc(newVertices * sizeof(int*));
    for (int i = 0; i < newVertices; i++) {
        newMatrix[i] = (int*)calloc(newVertices, sizeof(int));
    }
}

```

```

    }

    for (int i = 0; i < graph->vertices; i++) {
        for (int j = 0; j < graph->vertices; j++) {
            newMatrix[i][j] = graph->matrix[i][j];
        }
    }

    int newVertex = newVertices - 1;

    newMatrix[idx][newVertex] = 1;
    newMatrix[newVertex][idx] = 1;

    for (int i = 0; i < graph->vertices; i++) {
        if (graph->matrix[idx][i] && i != idx) {
            if (i % 2 == 0) {
                newMatrix[newVertex][i] = 1;
                newMatrix[i][newVertex] = 1;
                newMatrix[idx][i] = 0;
                newMatrix[i][idx] = 0;
            }
        }
    }

    for (int i = 0; i < graph->vertices; i++) {
        free(graph->matrix[i]);
    }
    free(graph->matrix);

    graph->matrix = newMatrix;
    graph->vertices = newVertices;
}

void createExampleGraph(GraphMatrix* graph) {
    for (int i = 1; i < graph->vertices; i++) {
        addEdgeMatrix(graph, i, i + 1);
    }
    if (graph->vertices > 1) {
        addEdgeMatrix(graph, 1, graph->vertices);
    }

    if (graph->vertices > 3) {
        addEdgeMatrix(graph, 1, 3);
        addEdgeMatrix(graph, 2, 4);
    }
}

void freeGraphMatrix(GraphMatrix* graph) {
    for (int i = 0; i < graph->vertices; i++) {
        free(graph->matrix[i]);
    }
    free(graph->matrix);
    free(graph);
}

int main() {
    system("chcp 1251");
    int v1, v2, operation;

```



```

printf("=== ОПЕРАЦИИ С ГРАФАМИ (МАТРИЧНАЯ ФОРМА) ===\n");

printf("Введите количество вершин в графе: ");
int vertices;
scanf("%d", &vertices);

if (vertices <= 0) {
    printf("Количество вершин должно быть положительным!\n");
    return 1;
}

GraphMatrix* graph = createGraphMatrix(vertices);
createExampleGraph(graph);

printf("\nИсходный граф:\n");
printGraphMatrix(graph);

while (1) {
    printf("\nВыберите операцию:\n");
    printf("1 - Отождествление вершин\n");
    printf("2 - Стыгивание ребра\n");
    printf("3 - Расщепление вершины\n");
    printf("0 - Выход\n");
    printf("Ваш выбор: ");
    scanf("%d", &operation);

    if (operation == 0) break;

    switch (operation) {
        case 1:
            printf("Введите две вершины для отождествления: ");
            scanf("%d %d", &v1, &v2);
            if (v1 < 1 || v1 > graph->vertices || v2 < 1 || v2 >
graph->vertices) {
                printf("Неверные номера вершин! Допустимы от 1 до
%d\n", graph->vertices);
            }
            else {
                printf("Отождествление вершин %d и %d \n", v1, v2);
                identifyVerticesMatrix(graph, v1, v2);
                printf("Результат отождествления:\n");
                printGraphMatrix(graph);
            }
            break;

        case 2:
            printf("Введите две вершины для стыгивания ребра: ");
            scanf("%d %d", &v1, &v2);
            if (v1 < 1 || v1 > graph->vertices || v2 < 1 || v2 >
graph->vertices) {
                printf("Неверные номера вершин! Допустимы от 1 до
%d\n", graph->vertices);
            }
            else {
                contractEdgeMatrix(graph, v1, v2);
                printf("Результат стыгивания ребра между вершинами %d
и %d:\n", v1, v2);
            }
            break;
    }
}

```

```

        printGraphMatrix(graph);
    }
    break;

    case 3:
        printf("Введите вершину для расщепления: ");
        scanf("%d", &v1);
        if (v1 < 1 || v1 > graph->vertices) {
            printf("Неверный номер вершины! Допустимы от 1 до
%d\n", graph->vertices);
        }
        else {
            splitVertexMatrix(graph, v1);
            printf("Результат расщепления вершины %d:\n", v1);
            printGraphMatrix(graph);
        }
        break;

    default:
        printf("Неверный выбор! Попробуйте снова.\n");
    }
}

freeGraphMatrix(graph);
printf("Программа завершена.\n");

return 0;
}

```

Задание 3.

Код программы

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void generateRandomMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else if (i < j) {
                matrix[i][j] = rand() % 2;
            }
            else {
                matrix[i][j] = matrix[j][i];
            }
        }
    }
}

void printMatrix(int** matrix, int vertices, const char* title) {
    printf("\n%s:\n", title);
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {

```

```

        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}

void unionGraphs(int** G1, int n1, int** G2, int n2, int** result, int
max_vertices) {
    for (int i = 0; i < max_vertices; i++) {
        for (int j = 0; j < max_vertices; j++) {
            int val1 = (i < n1 && j < n1) ? G1[i][j] : 0;
            int val2 = (i < n2 && j < n2) ? G2[i][j] : 0;
            result[i][j] = val1 || val2;
        }
    }
}

void intersectionGraphs(int** G1, int n1, int** G2, int n2, int**
result, int max_vertices) {
    for (int i = 0; i < max_vertices; i++) {
        for (int j = 0; j < max_vertices; j++) {
            int val1 = (i < n1 && j < n1) ? G1[i][j] : 0;
            int val2 = (i < n2 && j < n2) ? G2[i][j] : 0;
            result[i][j] = val1 && val2;
        }
    }
}

void ringSumGraphs(int** G1, int n1, int** G2, int n2, int** result,
int max_vertices) {
    for (int i = 0; i < max_vertices; i++) {
        for (int j = 0; j < max_vertices; j++) {
            int val1 = (i < n1 && j < n1) ? G1[i][j] : 0;
            int val2 = (i < n2 && j < n2) ? G2[i][j] : 0;
            result[i][j] = val1 ^ val2;
        }
    }
}

int** allocateMatrix(int vertices) {
    int** matrix = (int**)malloc(vertices * sizeof(int*));
    for (int i = 0; i < vertices; i++) {
        matrix[i] = (int*)calloc(vertices, sizeof(int));
    }
    return matrix;
}

void freeMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    system("chcp 1251");
    int n1, n2;

```

```

printf("Введите количество вершин графа G1: ");
scanf("%d", &n1);
printf("Введите количество вершин графа G2: ");
scanf("%d", &n2);

if (n1 <= 0) {
    printf("Ошибка: количество вершин должно быть положительным
числом.\n");
    return 1;
}
if (n2 <= 0) {
    printf("Ошибка: количество вершин должно быть положительным
числом.\n");
    return 1;
}

srand(time(NULL));

int max_vertices = (n1 > n2) ? n1 : n2;

int** G1 = allocateMatrix(n1);
int** G2 = allocateMatrix(n2);
int** result = allocateMatrix(max_vertices);

generateRandomMatrix(G1, n1);
generateRandomMatrix(G2, n2);

printMatrix(G1, n1, "Граф G1");
printMatrix(G2, n2, "Граф G2");

unionGraphs(G1, n1, G2, n2, result, max_vertices);
printMatrix(result, max_vertices, "Объединение G1 и G2");

intersectionGraphs(G1, n1, G2, n2, result, max_vertices);
printMatrix(result, max_vertices, "Пересечение G1 и G2");

ringSumGraphs(G1, n1, G2, n2, result, max_vertices);
printMatrix(result, max_vertices, "Кольцевая сумма G1 и G2");

freeMatrix(G1, n1);
freeMatrix(G2, n2);
freeMatrix(result, max_vertices);

return 0;
}

```

Задание 4.

Код программы

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int** createMatrix(int rows, int cols) {
    int** matrix = (int**)malloc(rows * sizeof(int*));

```

```

    for (int i = 0; i < rows; i++) {
        matrix[i] = (int*)malloc(cols * sizeof(int));
    }
    return matrix;
}

void freeMatrix(int** matrix, int rows) {
    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void generateRandomMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (i == j) {
                matrix[i][j] = 0;
            }
            else if (i < j) {
                matrix[i][j] = rand() % 2;
            }
            else {
                matrix[i][j] = matrix[j][i];
            }
        }
    }
}

void printMatrix(int** matrix, int rows, int cols, const char* name) {
    printf("\nМатрица %s (%dx%d):\n", name, rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int** cartesianProduct(int** G1, int n1, int** G2, int n2) {
    int resultSize = n1 * n2;
    int** result = createMatrix(resultSize, resultSize);

    for (int i = 0; i < resultSize; i++) {
        for (int j = 0; j < resultSize; j++) {
            result[i][j] = 0;
        }
    }

    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < n2; j++) {
            for (int k = 0; k < n1; k++) {
                for (int l = 0; l < n2; l++) {
                    int row = i * n2 + j;
                    int col = k * n2 + l;

                    if (i == k && G2[j][l] == 1) {
                        result[row][col] = 1;
                    }
                }
            }
        }
    }
}

```

```

        }
        else if (j == 1 && G1[i][k] == 1) {
            result[row][col] = 1;
        }
    }
}

return result;
}

void printCartesianVertices(int n1, int n2) {
    printf("\nВершины декартова произведения G1 X G2:\n");
    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < n2; j++) {
            printf("(%d,%d) ", i, j);
        }
        printf("\n");
    }
}

int main() {
    system("chcp 1251");
    srand(time(NULL));

    int n1, n2;

    printf("=== Декартово произведение графов G1 X G2 ===\n\n");

    printf("Введите количество вершин графа G1: ");
    scanf("%d", &n1);
    printf("Введите количество вершин графа G2: ");
    scanf("%d", &n2);

    int** G1 = createMatrix(n1, n1);
    int** G2 = createMatrix(n2, n2);

    generateRandomMatrix(G1, n1);
    generateRandomMatrix(G2, n2);

    printMatrix(G1, n1, n1, "G1");
    printMatrix(G2, n2, n2, "G2");

    int** result = cartesianProduct(G1, n1, G2, n2);
    int resultSize = n1 * n2;

    printCartesianVertices(n1, n2);

    printMatrix(result, resultSize, resultSize, "G1 X G2 (декартово
произведение)");

    freeMatrix(G1, n1);
    freeMatrix(G2, n2);
    freeMatrix(result, resultSize);

    return 0;
}

```

Результаты работы программ

```
Матрица смежности M1 (граф G1):
  1  2  3  4  5
1:  0  0  0  0  0
2:  0  0  1  1  1
3:  0  1  0  1  0
4:  0  1  1  0  0
5:  0  1  0  0  0

Матрица смежности M2 (граф G2):
  1  2  3  4  5
1:  0  0  1  0  0
2:  0  0  0  1  1
3:  1  0  0  0  1
4:  0  1  0  0  0
5:  0  1  1  0  0

=== ПРЕОБРАЗОВАНИЕ В СПИСКИ СМЕЖНОСТИ ===

Список смежности для графа G1:
1:
2: 5 4 3
3: 4 2
4: 3 2
5: 2

Список смежности для графа G2:
1: 3
2: 5 4
3: 5 1
4: 2
5: 3 2
```

Рисунок 1 – результат работы программы 1

```
Исходный граф:
Матрица смежности:
0 1 1 0 1
1 0 1 1 0
1 1 0 1 0
0 1 1 0 1
1 0 0 1 0

Выберите операцию:
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Ваш выбор: 1
Введите две вершины для отождествления: 1 2
Отождествление вершин 1 и 2
Результат отождествления:
Матрица смежности:
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0
```

Рисунок 2 – результат работы программы 2

```

Исходный граф:
Матрица смежности:
0 1 1 0 1
1 0 1 1 0
1 1 0 1 0
0 1 1 0 1
1 0 0 1 0

Выберите операцию:
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Ваш выбор: 2
Введите две вершины для стягивания ребра: 1 2
Результат стягивания ребра между вершинами 1 и 2:
Матрица смежности:
0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0

```

Рисунок 3 – результат работы программы 2

```

Исходный граф:
Матрица смежности:
0 1 1 0 1
1 0 1 1 0
1 1 0 1 0
0 1 1 0 1
1 0 0 1 0

Выберите операцию:
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Ваш выбор: 3
Введите вершину для расщепления: 3
Результат расщепления вершины 3:
Матрица смежности:
0 1 0 0 1 1
1 0 1 1 0 0
0 1 0 1 0 1
0 1 1 0 1 0
1 0 0 1 0 0
1 0 1 0 0 0

```

Рисунок 4 – результат работы программы 2


```

Введите количество вершин графа G1: 5
Введите количество вершин графа G2: 4

Граф G1:
0 0 1 0 0
0 0 1 0 1
1 1 0 1 1
0 0 1 0 1
0 1 1 1 0

Граф G2:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0

Объединение G1 и G2:
0 1 1 0 0
1 0 1 1 1
1 1 0 1 1
0 1 1 0 1
0 1 1 1 0

Пересечение G1 и G2:
0 0 1 0 0
0 0 0 0 0
1 0 0 1 0
0 0 1 0 0
0 0 0 0 0

Кольцевая сумма G1 и G2:
0 1 0 0 0
1 0 1 1 1
0 1 0 0 1
0 1 0 0 1
0 1 1 1 0

```

Рисунок 5 – результат работы программы 3

```

=== Декартово произведение графов G1 X G2 ===

Введите количество вершин графа G1: 3
Введите количество вершин графа G2: 2

Матрица G1 (3x3):
0 1 0
1 0 0
0 0 0

Матрица G2 (2x2):
0 1
1 0

Вершины декартова произведения G1 X G2:
(0,0) (0,1) (1,0) (1,1) (2,0) (2,1)

Матрица G1 X G2 (декартово произведение) (6x6):
0 1 1 0 0 0
1 0 0 1 0 0
1 0 0 1 0 0
0 1 1 0 0 0
0 0 0 0 0 1
0 0 0 0 1 0

```

Рисунок 6 – результат работы программы 4

Вывод: в ходе выполнения лабораторной работы изучили унарные и бинарные операций над графами, включая операции, уменьшающие или увеличивающие число элементов графа, такие как удаление и добавление вершин и рёбер, отождествление и расщепление вершин, стягивание рёбер.